

Distributing a Travel Reservation System

COMP 512 – Programming Assignment 1

Group 42: Arman Ahmed, Frédéric Lahaie-Bertrand

October 2026

Shared architecture

Structure

- Client is unchanged and only talks to the middleware
- Middleware routes each request to one of the three RMs (Flights, Cars, Rooms)
- Three RMs run the (unchanged) ResourceManager from the template
- Middleware only calls each RM's own resource methods

Customers

- Replicated on all RMs, each copy only holds resources associated with that RM
- **Benefits:** Reservation is a single call, failure causes only a partial loss of customers, minimal changes to code (RM and reserveltem unchanged)
- **Drawback:** creating, deleting and querying a customer involve all RMs

Bundles

- All-or-nothing, coordinated by middleware
- No way to cancel a single reservation, RMI and TCP handle this differently

- **Startup:** RMIMiddleware implements IResourceManager; Binds only once all three RMs are reachable
- **Exceptions:** RM's RemoteException not caught; propagated to the client as a ServerException and is printed as a command error
- **Customers:** IDs with sequential counter (in middleware); Failed creation undone on each RM where it succeeded; Failed deletion only reports to the client which failed; Total bill computed by middleware
- **Bundles:** Validate, pre-check, then reserve; Global lock; Rm crashing during phase 3 throws an exception that tells the client which reservations were made
- **Concurrency:** Global lock for all stateful operations; Timeouts to make sure lock is released even if a RM is hanging; Serializes the middleware but simpler
- **Negative counts:** Returns false

- Clients get their own persistent socket to the middleware (implemented in TCPChannel)
- Client reads a command and calls TCPResourceManagerClient which invokes the command, sending it to the middleware.
- The middleware has a dedicated thread for each client. When clients connect the TCPMiddlewareServer creates this thread and gives it a ResourceManagerDispatcher.
- This dispatcher is the interface between the middleware and TCPResourceManagerServer. It manages command dispatch and state locking (read lock, write lock)
 - Read operations can occur concurrently, mutations take the write lock, and occur one at a time.
- The TCPResourceManagerServer does the computation and sends back a response to the middleware, which sends it back to the client.
- We use classes Response, Request to send and receive messages, taking advantage of java's object serialization. This is fully general and easily extensible.